

Here's a breakdown of the Contabo **Secrets Management API**—what it is, how to use it, and what to watch out for—so you (as an IT Manager) can evaluate whether it fits into your workflow.

 What it is

The “Secrets” API is a part of Contabo’s larger cloud-resources API. It allows you to store and manage *sensitive items* such as SSH public keys or passwords that you might reuse across servers. From the docs:

- It’s optional: you don’t *have* to use it to operate resources. [api.contabo.com](#) +2
- It supports operations like:
 - Create a secret (`POST /v1/secrets`) [GitHub](#) +2
 - Retrieve a list of secrets (`GET /v1/secrets`) [GitHub](#) +1
 - Retrieve details of a secret by ID (`GET /v1/secrets/{secretId}`) [GitHub](#)
 - Update a secret (`PATCH /v1/secrets/{secretId}`) [GitHub](#)
 - Delete a secret (`DELETE /v1/secrets/{secretId}`) [GitHub](#)

So, it is essentially a lightweight secret-store associated with your Contabo account, intended for internal reuse (e.g., reuse SSH keys between VMs) rather than a full-blown secrets management platform.

 How to use it

Here's a practical outline you'd follow to integrate it (in a broad sense):

- 1. Obtain credentials**
- In the Contabo Control Panel: find your Client ID, Client Secret, API User (your email) and set an API Password. [api.contabo.com](#) +1
 - Use those to generate an access token via OAuth2 (Contabo uses the `grant_type=password` flow). Example from the docs:

bash  Copy code

```
CLIENT_ID=<ClientId>
CLIENT_SECRET=<ClientSecret>
API_USER=<your email>
API_PASSWORD=<your api password>
ACCESS_TOKEN=$(curl -d "client_id=$CLIENT_ID" \
                    -d "client_secret=$CLIENT_SECRET" \
                    --data-urlencode "username=$API_USER" \
                    --data-urlencode "password=$API_PASSWORD" \
                    -d 'grant_type=password' \
                    'https://auth.contabo.com/auth/realms/contabo/protocol/openid-c
| jq -r '.access_token')
```

[api.contabo.com](#)

- 2. Invoke the Secrets API**
- Using the base URL `https://api.contabo.com` (for version v1). [GitHub](#) +1
 - Add header `Authorization: Bearer <ACCESS_TOKEN>`
 - Add an `X-Request-ID` header (a UUID4) to identify your request. Example from docs:
`x_request_id = '04e0f898-37b4-48bc-a794-1a57abe6aa31'` . [GitHub](#)
 - Example: create a secret (request body must include `name` , `type` (either `password` or `ssh`), `value`). [GitHub](#)

- 3. Example in code** (Python sample from docs):

python  Copy code

```
import pfruck_contabo
from pfruck_contabo.models.create_secret_request import CreateSecretRequest
```

```

configuration = pfruck_contabo.Configuration(
    host = "https://api.contabo.com",
    access_token = os.environ["BEARER_TOKEN"]
)

with pfruck_contabo.ApiClient(configuration) as api_client:
    api_instance = pfruck_contabo.SecretsApi(api_client)
    x_request_id = 'uuid4-here'
    create_secret_request = CreateSecretRequest(
        name="mySSHKey",
        type="ssh",
        value="ssh-rsa AAAAB3NzaC1..."
    )
    response = api_instance.create_secret(x_request_id, create_secret_request)
    print(response)

```

GitHub +1

⚠ Things to watch / limitations

Since you're in IT management, these are risk/regime considerations:

- **Scope & features:** this is *not* a full secrets lifecycle management platform (e.g., versioning, dynamic retrieval, rotation, etc.). It's simpler: store SSH keys / passwords for reuse.
- **Security practices:** Even though it's "secrets", you should treat them like any sensitive credential: limit access, audit retrievals, ensure encryption at rest, etc.
- **Access control:** You'll want to control *who* (which user/API role) can create/read/update/delete secrets. The API does indicate user and role management endpoints. [api.contabo.com](#) +1
- **Integration with infrastructure:** If you have an environment with automation (e.g., Terraform, CI/CD), check how well it integrates (for instance, there's a Terraform resource `contabo_secret`).
[Terraform Registry](#)
- **Audit & logging:** Ensure your usage generates logs (e.g., `X-Request-ID` , `X-Trace-ID` fields exist) so you can trace operations.
- **Secrets retrieval:** Confirm how the "value" returns (plaintext or masked) and how it is transmitted over the wire — you should make sure it's always over TLS and ideally not stored in logs.

🧩 How this fits *your* stack

Since you are an IT Manager, here are some potential use-cases and how you might leverage this:

- You have multiple VMs on Contabo and need to reuse SSH public keys — rather than manually tracking them, you store them once via the Secrets API and reference them when provisioning.
- You want an internal pipeline that pulls "passwords" or "keys" from a controlled store when spinning up servers. Using this API means the secret store is within Contabo's ecosystem (no separate vault).
- You're deciding if you need a full secrets management product (like HashiCorp Vault, AWS Secrets Manager) vs a simpler built-in Contabo secret store. If your requirements are light (just reuse keys/passwords) the built-in may suffice.

If you like, I can pull **the full OpenAPI spec** of the Secrets endpoint for you (with all fields, response codes etc), so you can plug it into your API catalogue. Would that be helpful?

📄 👍 💬 ⬆ ↺ ... 🌐 🐙 Sources